

Guia de Preparação para o Seminário

Trabalho de IA — Torre de Hanói (UCS, Busca Gulosa e A*)

Kauã Cardoso Benini, Gabriel Benini e Felipe Donatoni

1. As regras do seminário (não esquecer)

- **Duração:** 10 a 15 minutos, + 5 minutos de perguntas.
- **Os três precisam falar.** Não pode um ou dois carregarem a apresentação sozinhos.
- **Não é aula de teoria.** Não expliquem "o que é UCS" ou "o que é A*" do zero — o professor já deu isso em aula. O que ele quer é o **relato da experiência de vocês construindo**.
- **Qualquer um pode ser questionado sobre qualquer linha do código.** Quem não souber explicar tem a nota individual reduzida — mesmo que o código esteja perfeito.
- **Levem o programa pronto pra rodar.** Falha técnica na hora não é desculpa.
- **Um erro real, bem explicado, vale mais nota** do que dizer "deu tudo certo de primeira".

2. Divisão sugerida entre vocês três

Esse guia agora segue **a mesma ordem dos 14 slides** de `slides_hanoi.pptx` — cada bloco da Seção 4 corresponde a um slide (ou par de slides), na mesma sequência.

Quem	Slides	Blocos (Seção 4)
Pessoa A (ex.: Kauã)	1-2, 3, 4, 5	1. Título + O problema · 2. A escolha · 3. Modelagem · 4. Regras de movimento
Pessoa B (ex.: Gabriel)	6, 7, 8, 9	5. A regra mais importante · 6. Heurística · 7. Heurística fraca · 8. Resultados (tabela)
Pessoa C (ex.: Felipe)	10, 11, 12, 13-14	9. Resultados (gráfico) · 10. O imprevisto da Gulosa · 11. Quando usar cada um + demonstração ao vivo · 12. Conclusão + Obrigado

Ficou parelho: 4 blocos pra cada um, ~3-4 min por pessoa. Ajustem entre vocês se quiserem equilibrar melhor — o que importa é que **cada um sabe explicar os blocos dos outros também**, porque as perguntas do final podem ir pra qualquer um sobre qualquer parte, inclusive sobre o código (`hanoi.py`, `busca.py`, `resolver.py`, `gui.py`), não só sobre os slides.

3. O que cada um PRECISA entender (não decorar — entender)

3.1 A ideia central do trabalho, em uma frase

"UCS, Busca Gulosa e A* são a mesma busca (mesmo algoritmo de fila de prioridade), só mudando qual número usamos para ordenar a fila: $g(n)$ (custo já pago), $h(n)$ (palpite do que falta), ou os dois somados." Isso está em `busca.py`, na função `busca(problema, funcao_f, ...)`.

3.2 Por que Torre de Hanói é um bom problema pra esse trabalho

A Torre de Hanói tem uma vantagem que quase nenhum outro problema da lista tem: a solução ótima é conhecida por uma **fórmula matemática exata**, $2^n - 1$ movimentos para n discos. Isso significa que vocês não precisam "confiar" que a busca está certa — dá pra comparar o resultado com um gabarito exato em cada teste. Foi esse o motivo principal da escolha.

3.3 Como o estado é representado

Uma tupla de N posições, uma por disco (disco 0 é o menor). O valor em cada posição é o pino (0=A, 1=B, 2=C) onde aquele disco está — por exemplo, `(0, 0, 2, 1)` com 4 discos: os discos 1 e 2 (os dois menores) estão no pino A, o disco 3 está no pino C, o disco 4 está no pino B. É tupla, e não lista, porque o estado precisa ser **hashable** pra entrar num dicionário de "estados já vistos" (senão a busca fica reexpandindo a mesma posição pra sempre).

3.4 A regra de movimento, resumida

Em cada pino, só o disco de **menor índice** (o de cima da pilha) pode ser movido — nunca um disco maior pode ficar sobre um menor. Um disco só pode ir para outro pino se esse pino estiver vazio, ou se o disco que já está no topo dele for maior.

3.5 Como funciona a heurística, e por que ela é fraca

A heurística é a que a própria ficha do problema sugere: **quantos discos ainda não estão no pino destino**. Ela é admissível porque cada disco fora do lugar vai precisar de pelo menos 1 movimento antes do fim — então o número que a heurística devolve nunca é maior que o número real de movimentos que faltam.

Mas ela é **fraca de propósito**: ela trata igual um disco que só precisa de 1 movimento pra chegar no destino e um disco que está "enterrado" sob vários discos maiores e vai precisar de muitos movimentos antes de sequer poder se mexer. Os dois contam como "1 disco fora do lugar" — por isso ela guia a busca menos do que se poderia esperar.

3.6 O resultado mais interessante para contar

Em todos os testes (3 a 10 discos), os três algoritmos **sempre bateram exatamente com o gabarito da fórmula** ($2^n - 1$) — nenhum errou o custo, nem a Gulosa. Mas, olhando os nós expandidos, a Gulosa nem sempre foi mais eficiente que o A*: em alguns casos ela expandiu menos nós, em outros, mais (por exemplo, com 10 discos, a Gulosa expandiu 62.080 nós contra 55.881 do A*). Isso mostra que uma heurística fraca ainda garante corretude (porque é admissível), mas não garante uma vantagem de eficiência consistente.

4. Roteiro passo a passo (um bloco por slide)

Cada bloco tem o **número do slide**, uma **fala sugerida** (não precisa decorar igual, é só pra vocês verem o tamanho certo de conteúdo pro tempo) e, quando cabe, um **"se sobrar tempo"** pra esticar naturalmente. Troquem de slide exatamente na hora que o bloco muda.

Bloco 1 — Slides 1-2: Título e "O problema" (~1,5-2 min)

Fala sugerida: "Boa tarde/noite, somos o Kauã, o Gabriel e o Felipe, e vamos apresentar o trabalho de Inteligência Artificial sobre comparação de UCS, Busca Gulosa e A*. Escolhemos o Problema 2 da lista — Torre de Hanói.

(troca pro slide 2) O problema consiste em mover uma pilha de discos de tamanhos diferentes de um pino pra outro, usando um pino auxiliar, sem nunca colocar um disco maior sobre um menor. São três pinos — A, onde tudo começa, B, que só ajuda a passagem, e C, o destino final. Parece simples, mas por trás dessa regra única mora uma vantagem rara que vamos explorar: a solução ótima tem uma fórmula matemática exata, dois elevado a n menos um movimentos, o que dá um gabarito pra conferir cada teste sem precisar só confiar visualmente."

Se sobrar tempo: comentem que a Torre de Hanói também aparece como um problema "clássico" de recursão em ciência da computação, e que a solução recursiva conhecida (mover n-1 discos, mover o maior, mover os n-1 de volta) é o que gera exatamente a fórmula $2^n - 1$ — mas o trabalho pedia resolver via busca, não via recursão direta.

Bloco 2 — Slide 3: "A escolha" (~1-1,5 min)

Fala sugerida: "Escolhemos esse problema por três motivos, que são os três pontos do slide. Primeiro, o gabarito exato: se o código errasse, a gente saberia na hora, comparando com dois elevado a n menos um — não precisávamos só confiar visualmente no resultado. Segundo, o espaço de estados cresce como três elevado a n, então a explosão combinatória que o trabalho quer que a gente enxergue fica bem visível na prática, não só na teoria. E terceiro, as regras são curtas — poucas linhas pra decorar — então os três conseguimos defender o código inteiro no seminário, sem nada escondido."

Bloco 3 — Slide 4: "Modelagem" (~1,5-2 min)

Fala sugerida: "Pra modelar o problema, representamos o estado como uma tupla de N posições, uma por disco, numerando os discos de 0, o menor, até N-1, o maior. O valor guardado em cada posição é o pino onde aquele disco está — 0 para A, 1 para B, 2 para C. No exemplo do slide, com 4 discos, a tupla (0, 0, 2, 1) significa que os discos 1 e 2 estão no pino A, o disco 3 já está no C, e o disco 4 está no B.

Usamos tupla e não lista pelo mesmo motivo de sempre nesse tipo de busca: o estado precisa ser hashable, ou seja, imutável, pra poder entrar como chave num dicionário — é assim que controlamos, no `busca.py`, quais estados já foram visitados, pra não reexpandir a mesma posição duas vezes. O custo de cada movimento é 1, igual pra qualquer disco, e o objetivo é todos os discos chegarem ao pino C."

Bloco 4 — Slide 5: "Regras de movimento" (~1,5 min)

Fala sugerida: "As ações do problema são só duas regras. Primeiro: só o disco do topo de um pino pode se mover — o de menor índice presente ali, porque um estado válido nunca tem disco maior sobre um menor. Segundo: esse disco só pode ir pra um pino vazio, ou pra um pino cujo disco do topo seja maior que ele. No slide, o cartão da esquerda mostra o caso permitido — disco pequeno sobre disco grande — e o da direita mostra o proibido, que a nossa função `movimentos_legais`, no `hanoi.py`, simplesmente nunca gera como opção."

Bloco 5 — Slide 6: "A regra mais importante" (~2-2,5 min)

Fala sugerida: "Esse é o ponto mais importante do trabalho inteiro. UCS, Busca Gulosa e A* não são três algoritmos diferentes no nosso código — são a mesma função de busca, no `busca.py`, chamada três vezes trocando só um parâmetro, o `f`. `g(n)` é quanto já custou chegar até aquele estado. `h(n)` é quanto a heurística acha que ainda falta. A UCS usa `f` igual a `g`, a Gulosa usa `f` igual a `h`, e o A* usa `f` igual à soma dos dois. Não existe nenhuma busca copiada e colada — é literalmente a regra que o enunciado do trabalho mais cobra."

Bloco 6 — Slide 7: "Heurística" (~1,5-2 min)

Fala sugerida: "A heurística que usamos é a que a própria ficha do problema sugere: `h(n)` igual ao número de discos que ainda não estão no pino destino. Ela é admissível porque cada disco fora do lugar certo vai precisar de pelo menos um movimento antes do fim da solução — então esse número nunca é maior do que os movimentos que realmente faltam. E confirmamos isso na prática: o A*, que usa essa heurística, nunca divergiu do gabarito em nenhum dos 24 testes que rodamos."

Bloco 7 — Slide 8: "Heurística (continuação) — fraca, de propósito" (~1,5-2 min)

Fala sugerida: "Só que essa heurística é fraca, de propósito. Ela trata igual um disco que só precisa de 1 movimento pra chegar no destino — o cartão da esquerda — e um disco 'enterrado' sob vários discos maiores, que vai precisar de muitos movimentos antes de sequer poder se mover — o cartão da direita. Os dois contam como '1 disco fora do lugar', exatamente igual. Isso limita o quanto ela consegue guiar a busca de forma eficiente, e é algo que vocês vão ver acontecer nos resultados."

Bloco 8 — Slide 9: "Resultados (tabela)" (~2 min)

Fala sugerida: "Testamos com 3 a 10 discos, como a própria ficha do problema sugeria. Em todos os 24 testes — 8 quantidades de discos vezes 3 algoritmos — o custo encontrado bateu exatamente com o gabarito da fórmula. Essa foi a primeira confirmação de que a implementação está correta. E já dá pra reparar na tabela: a coluna da UCS cresce mais rápido que as de Gulosa e A*, mesmo os três empatando em custo — a diferença aparece só na eficiência da busca, não na qualidade da resposta."

Bloco 9 — Slide 10: "Resultados (gráfico)" (~1,5-2 min)

Fala sugerida: "Esse gráfico mostra só a UCS, pra deixar bem claro o formato da curva: cada disco a mais multiplica o número de nós expandidos por aproximadamente três, coerente com o espaço de estados, que cresce como três elevado a `n`. Com três discos, vinte e quatro nós;

com dez discos, já são cinquenta e oito mil. Essa é a explosão combinatória que o trabalho quer que a gente enxergue na prática, e não só na teoria."

Se sobrar tempo: contem que testaram informalmente até doze discos fora do benchmark oficial, e que nesse ponto a UCS já levava mais de vinte e três segundos, contra bem menos tempo de Gulosa e A* — reforçando ainda mais a diferença entre busca cega e busca guiada.

Bloco 10 — Slide 11: "O imprevisto" (~2-2,5 min)

Fala sugerida: "Esse é o resultado mais interessante do trabalho. Com nove discos, a Gulosa expande menos nós que os outros dois — dezessete mil seiscentos e quarenta e dois, contra dezenove mil e quinhentos e doze da UCS. Mas com dez discos acontece o contrário: a Gulosa passa a expandir mais que ambos, sessenta e dois mil e oitenta contra cinquenta e cinco mil oitocentos e oitenta e um do A*. Isso acontece porque a heurística é fraca — ela guia pouco a busca, então às vezes o cálculo extra do A*, que soma custo já pago com o palpite do que falta, não compensa com uma economia real de nós. Mesmo assim, é importante notar que a Gulosa nunca chegou a errar o custo da solução em nenhum teste — só a eficiência dela que não é consistente."

Bloco 11 — Slide 12: "Quando usar cada um" + demonstração ao vivo (~3-3,5 min)

Fala sugerida: "Na prática, cada algoritmo tem seu lugar. UCS, quando não existe heurística confiável, mas o custo do caminho precisa ser garantidamente ótimo. Busca Gulosa, quando velocidade importa mais que otimalidade, e uma resposta boa o suficiente já serve. E A*, quando existe uma heurística admissível e se quer os dois: otimalidade com alguma economia de esforço."

Esse slide não tem mais um slide próprio de demonstração — a partir daqui, **saiam da apresentação e rodem o código ao vivo**, direto do terminal:

```
python main.py
```

Enquanto roda, narrem: "aqui está o benchmark completo rodando de novo, com os mesmos números que vocês viram nos slides, e na sequência uma demonstração com quatro discos, mostrando os quinze movimentos que o A* encontrou." Se der algum erro na hora, comentem com naturalidade e mostrem um print de resultado anterior como backup — mas tentem rodar ao vivo mesmo, é o que o roteiro do professor pede explicitamente.

Se sobrar tempo: rodem também `python gui.py`, resolvam com 4 ou 5 discos e cliquem em Animar — mostrem o botão de Pausar funcionando (ele cancela o próximo passo agendado e retoma exatamente do mesmo ponto) e o comparativo de nós expandidos dos três algoritmos, atualizado ao vivo, do lado do tabuleiro.

Bloco 12 — Slides 13-14: "Conclusão" e "Obrigado" (~1,5-2 min)

Fala sugerida: "Se recommêssemos o trabalho, duas coisas que faríamos diferente: testar uma heurística mais forte, que capturasse a estrutura recursiva do problema em vez de tratar '1 disco fora' sempre igual; e rodar mais cenários de benchmark, não só de 3 a 10 discos, pra ver se o cruzamento da Gulosa se repete em outros tamanhos. A interface gráfica opcional,

que vale dez por cento a mais na nota, a gente já entregou — vocês acabaram de ver ela rodando.

No fim, o maior aprendizado não foi programar os três algoritmos — isso é praticamente o mesmo código, só trocando uma função, o f do slide 6. O aprendizado foi perceber que uma heurística pode estar matematicamente certa, ser admissível, e ainda assim ajudar pouco, dependendo de quanto ela realmente representa a estrutura do problema.

(troca pro slide 14) Obrigado! Ficamos à disposição pra perguntas."

5. Perguntas prováveis do professor (e como responder)

"Por que tupla e não lista para representar o estado?" Porque o dicionário de "estados já visitados" (em `busca.py`) usa o estado como chave — e só valores *hashable* (imutáveis) podem ser chave de dicionário em Python. Lista é mutável, então dá erro `unhashable type`.

"Qual a diferença entre nó gerado e nó expandido?" Nó **gerado** é todo estado criado e colocado na fronteira (fila de prioridade). Nó **expandido** é o que foi **retirado** da fronteira e teve seus filhos gerados. A métrica que interessa pro trabalho é a de expandidos — é ela que mede o "esforço" real da busca.

"Por que a heurística de vocês é admissível?" Porque cada disco fora do pino destino precisa de pelo menos 1 movimento antes do fim da solução — então contar quantos discos estão fora do lugar nunca é maior que o número real de movimentos restantes. Na prática, confirmamos isso porque o A* nunca divergiu do gabarito conhecido ($2^n - 1$) em nenhum teste.

"Por que essa heurística é considerada fraca?" Porque ela não diferencia um disco que precisa de 1 movimento de um disco "enterrado" que precisa de muitos — ambos contam como "1 disco fora do lugar" igualmente. Ela ignora a estrutura recursiva do problema.

"Por que a Gulosa às vezes expandiu mais nós que o A*, se a Gulosa devia ser mais rápida?" Porque a heurística fraca guia pouco a busca — não existe garantia de que ignorar o custo já pago (como a Gulosa faz) sempre leve a menos nós expandidos. Isso depende de quão informativa é a heurística, e a nossa, apesar de correta, é deliberadamente simples.

"Por que a UCS se comporta como busca em largura aqui?" Porque todo movimento custa exatamente 1. Quando todos os custos são iguais, expandir por menor `g(n)` é a mesma coisa que expandir por profundidade crescente — que é exatamente o que a busca em largura faz.

"Por que escolheram testar até 10 discos e não mais?" Porque a partir de 10-12 discos o tempo já cresce bastante (a UCS passou de 23 segundos com 12 discos) devido à explosão combinatória do espaço de estados (3^n). Escolhemos uma faixa que mostrasse claramente a tendência de crescimento sem tornar o benchmark lento demais de rodar e apresentar.

"Como vocês sabem que a implementação está certa?" Porque a Torre de Hanói tem uma vantagem rara: a solução ótima é conhecida por fórmula matemática exata ($2^n - 1$). Em todos os 24 testes que rodamos (8 quantidades de discos $\times$ 3 algoritmos), o custo encontrado bateu exatamente com essa fórmula.

6. Checklist final antes de apresentar

- [] Rodar `python main.py` pelo menos uma vez no dia da apresentação, pra confirmar que continua funcionando na máquina que vocês vão usar.
- [] Rodar `python gui.py` também, resolver com uns 4-5 discos, clicar em Animar e testar o botão Pausar — se for usar no "se sobrar tempo" do Bloco 11.
- [] Ter `slides_hanoi.pptx` (ou o `.pdf`) aberto e testado na máquina que vão usar — não dependem do site do Claude nem de internet pra abrir.
- [] Ter o VS Code aberto já na pasta `TrabalhoIA_Hanoi`, com `main.py`, `busca.py`, `resolver.py`, `hanoi.py` e `gui.py` em abas prontas para mostrar.
- [] Ter o relatório em PDF aberto (ou impresso), pra apontar a tabela de resultados durante a fala.
- [] Combinar quem fala em qual bloco (Seção 2 deste guia) e treinar a transição pro terminal no Bloco 11 (sair dos slides, rodar `python main.py`, voltar pro slide 13).
- [] Rer a Seção 5 deste guia (perguntas prováveis) — não precisa decorar, só entender a lógica de cada resposta.